

Keywords: reinforcement learning • RLVR • reasoning diversity • pass@k • LLM alignment

When RLVR Shrinks the Reasoning Boundary: Diagnosing Pass@k Inversion

Reinforcement Learning with Verifiable Rewards Boosts One-Shot Accuracy While Silently Collapsing Sampling Diversity—a Failure Mode Traced to Boundary Prompts Where Rare Correct Trajectories Vanish Before They Can Be Reinforced

By Todd Zhou

arXiv:2607.20543 • July 2026

Reinforcement learning with verifiable rewards (RLVR) has become the dominant recipe for training reasoning models, and its track record at improving pass@1 is well-documented. A new preprint reveals the other side of the ledger: RLVR can simultaneously *decrease* pass@k for large k , meaning the trained model solves fewer distinct problems under repeated sampling than the base model it was fine-tuned from. The author terms this phenomenon **pass@k inversion** and traces it to a concrete failure mode involving rare correct trajectories.

AT A GLANCE

Problem: RLVR is evaluated almost exclusively at pass@1, masking the fact that it can reduce coverage under diverse sampling.

Why it matters: Any system using best-of- N , voting, or repeated sampling to boost reliability may be *harm*ed by RLVR training on boundary prompts.

Method: Pass@k evaluated pre- and post-RLVR across MATH and GSM8K; boundary prompts identified by base model entropy; causal rollout analysis tracing the loss of rare correct trajectories.

Result: RLVR improves pass@1 by 5–12 pp while reducing pass@16 by 3–8 pp on 15–25% of test problems.

Evidence: Per-Problem Base Anchoring (PBA) recovers 60% of the pass@k loss while preserving 95% of the pass@1 gains.

The Big Picture

Pass@k is a standard metric for reasoning benchmarks: it asks what fraction of problems a model can solve if given k independent sampling attempts, and a correct answer in any one attempt counts. For a fixed pass@1 score, higher pass@k reflects richer exploration of the solution space—a property valuable when models are used

with sampling-based inference strategies such as best-of- N selection, majority voting, or Monte Carlo Tree Search.

The intuition is that if RLVR improves a model, it should improve both pass@1 and pass@k. This paper shows the intuition is wrong for a non-trivial fraction of problems. After RLVR, the policy is sharper—it assigns higher probability to a narrower set of trajectories—and this sharpening can eliminate rare-but-correct paths that the base model would occasionally find at large k .

Why Existing Approaches Fall Short

Standard RLVR training (e.g., GRPO) collects G rollouts per problem, assigns binary rewards, and updates the policy to increase the relative likelihood of correct rollouts over incorrect ones. This mechanism has an implicit vulnerability: if none of the G rollouts for a problem are correct, the update treats the problem as definitively unsolvable and suppresses all trajectories toward its vicinity in policy space.

For **boundary prompts**—problems where the base model’s correct-trajectory probability is, say, 5–15%—a rollout group of size $G = 8$ will observe zero correct rollouts in a large fraction of training encounters. Each such encounter applies a gradient that reduces the probability of rare correct trajectories, eventually driving them to near zero even though they were genuinely present.

Core Mechanism

The author proposes a *two-mode account* of post-RLVR policy distributions:

- **Safe prompts:** Base model has high enough correct-trajectory probability that at least one correct rollout appears reliably in each group. RLVR reinforces and concentrates this probability. Pass@k improves monotonically with pass@1.
- **Boundary prompts:** Correct trajectories are too sparse to survive finite rollout groups. RLVR receives an absence-of-evidence signal and drives correct-trajectory probability toward zero. Pass@k *collapses* even as pass@1 may improve on adjacent safe prompts in the same batch.

Formally, for a boundary prompt x with base policy correct probability $\pi_{\text{base}}(x) \in [\epsilon, \delta]$ (small $\epsilon, \delta > 0$), the expected number of correct rollouts in a group of size G is $G \cdot \pi_{\text{base}}(x)$. When this falls below 1, the probability of observing zero correct rollouts in the group is approximately $(1 - \pi_{\text{base}})^G$, which for $\pi_{\text{base}} = 0.08$ and $G = 8$ is 50.6%.

Technical Formulation

Define pass@k for a single problem x and policy π as:

$$\text{pass@}k(x, \pi) = 1 - (1 - \pi_{\text{correct}}(x))^k$$

where $\pi_{\text{correct}}(x) = \sum_{a \in \mathcal{A}(x)} \pi(a | x)$ is the policy’s total mass on correct answers. Pass@ k inversion on problem x occurs when:

$$\pi_{\text{RL,correct}}(x) < \pi_{\text{base,correct}}(x)$$

even if $\pi_{\text{RL,correct}}(x) > \pi_{\text{base,correct}}(x)$ holds *on average* across all problems.

The GRPO update applied to a rollout group $\{r_1, \dots, r_G\}$ where all r_i are incorrect is:

$$\nabla \mathcal{L}_{\text{GRPO}} \propto - \sum_{i=1}^G \nabla \log \pi_{\theta}(r_i | x)$$

which suppresses the probability of all sampled trajectories. Since rare correct trajectories were not sampled, they are inadvertently suppressed via policy smoothing.

Learning or Inference Procedure

The Per-Problem Base Anchoring (PBA) fix proceeds as follows:

1. Classify each training problem as boundary or safe using the base model’s entropy $H(\pi_{\text{base}}(\cdot | x))$; problems above a threshold H^* are classified as boundary.
2. For safe problems, apply standard RLVR with no modification.
3. For boundary problems, augment the rollout group with forced correct samples from the frozen base model before computing the RLVR gradient.
4. Additionally, apply a KL penalty on boundary problems:

$$\mathcal{L}_{\text{PBA}} = \mathcal{L}_{\text{RLVR}} + \lambda D_{\text{KL}}(\pi_{\text{base}} \| \pi_{\theta})$$

which anchors the policy toward the base distribution on these inputs.

What the Guarantee Says

PBA does not guarantee recovery of all lost pass@ k on boundary prompts. Rather, it guarantees that the policy cannot reduce correct-trajectory probability below a KL-bounded neighborhood of the base policy. For $\lambda > 0$ and $D_{\text{KL}}(\pi_{\text{base}} \| \pi_{\theta}) \leq \epsilon_{\lambda}$, the trained policy satisfies:

$$\pi_{\text{RL,correct}}(x) \geq \pi_{\text{base,correct}}(x) \cdot e^{-\epsilon_{\lambda}}$$

providing a fractional lower bound on retained diversity.

Experimental Findings

Experiments use Llama-3 8B and Qwen2.5 7B as base models, with GRPO-style RLVR on MATH Level 3–5 and GSM8K problems.

Model	Metric	Base	+RLVR
Qwen2.5-7B	pass@1	56.3	68.1
Qwen2.5-7B	pass@16	83.2	78.4
Llama-3-8B	pass@1	48.9	57.3
Llama-3-8B	pass@16	79.1	73.6

The pass@16 regressions are concentrated on boundary prompts (15–25% of the test set) and are negligible on safe prompts. PBA recovers 60% of the pass@ k regression on boundary prompts while retaining 95% of the pass@1 gain.

Ablations and Interpretation

Group size G : Larger rollout groups ($G = 16$ vs. $G = 8$) reduce but do not eliminate inversion, as boundary prompts can still yield zero correct rollouts when $\pi_{\text{base,correct}}(x) < 1/G$.

Problem difficulty: Pass@ k inversion is uncorrelated with pass@1 improvement magnitude; problems that yield large pass@1 gains may or may not be on boundary prompts.

Anchor weight λ : Higher λ recovers more pass@ k but reduces pass@1 gains. The optimal λ is task-dependent.

Broader implication: Evaluating reasoning models solely at pass@1 is insufficient for any deployment context using sampling-based inference. The community should adopt pass@ k evaluation as standard alongside pass@1 when reporting RLVR results.

Reference

Todd Zhou. **When RLVR Shrinks the Reasoning Boundary: Diagnosing Pass@ k Inversion.** arXiv:2607.20543, July 2026. <https://arxiv.org/abs/2607.20543>

Keywords: token compression • context distillation • LLM efficiency • attention mechanisms

Progressive Cramming: Reliable Token Compression and What It Reveals

A Token-by-Token Compression Protocol Uncovers a Fundamental Capacity Limit in LLM Embedding Space—and Traces Downstream Task Degradation to Destructive Early-Layer Attention Interactions

By Dmitrii Tarasov, Timofei Lashukov, Elizaveta Goncharova, Andrey Kuznetsov

arXiv:2607.21231 • July 23, 2026

Token cramming—compressing long input prefixes into a small set of learned continuous embeddings—promises to let LLMs reference long contexts at the cost of a handful of extra tokens. Prior work reports near-perfect reconstruction (99%+), but fixes both the token budget and the accuracy threshold, leaving a crucial ambiguity: is residual error an optimization shortfall, or a fundamental limit of the embedding space? This paper answers with **progressive cramming**, a protocol that grows the compression target token-by-token until failure, revealing hard content-dependent limits that optimization cannot breach.

AT A GLANCE

Problem: Existing token cramming evaluates at fixed budgets and thresholds, conflating optimization failure with fundamental compression limits.

Why it matters: Systems relying on token cramming for long-context compression may degrade silently at inference; knowing true compression horizons is essential for safe deployment.

Method: Progressive cramming grows target prefix token-by-token, stopping at reconstruction failure; causal attention-knockout interventions locate the degradation source.

Result: Crammed embeddings cause 4–9 pp accuracy drops on multiple-choice tasks and near-total collapse on generative tasks, traced to early-layer attention interactions.

Evidence: Compression horizons range from 28–74% of prefix length depending on content, revealing fundamental limits independent of optimizer tuning.

The canonical approach to long-context LLM inference is to extend the context window—but this incurs quadratic attention cost. Token cramming offers an alternative: compress a prefix of N tokens into $M \ll N$ learned embedding vectors, insert them into the context, and let the model proceed as if the prefix were present. Studies have shown these crammed embeddings can achieve > 99% token-level reconstruction accuracy, fueling optimism about practical deployment.

This paper asks a harder question: does reconstruction accuracy predict task performance? And is there a hard ceiling on how many tokens can be crammed, independent of how long you optimize? The answers are “no” and “yes”, respectively, with implications for any deployment of cramming-based compression.

Why Existing Approaches Fall Short

Standard cramming evaluates quality at a fixed compression ratio (e.g., compress 512 tokens into 32 embeddings) and declares success when reconstruction accuracy exceeds 99%. This methodology has two gaps:

Conflation of failure modes. If reconstruction fails at a given ratio, it is unclear whether (a) more optimization steps would succeed or (b) the information exceeds the embedding’s representational capacity. Distinguishing these requires a protocol that rules out optimization failure as an explanation.

Reconstruction $\neq$ task performance. A crammed embedding that faithfully reconstructs a prefix token-by-token may still disrupt downstream task performance if the embedding interferes with the model’s internal processing—even when the original prefix is also provided in context.

Core Mechanism

Progressive cramming resolves the first gap by progressively growing the target prefix from length 1 to length N , applying a fixed optimization budget at each step. At length n , it finds the best embedding for the first n tokens. At some critical length n^* , optimization fails to achieve a fixed reconstruction threshold within the budget. Because the budget is held constant throughout, failure at n^* cannot be attributed to insufficient compute—it reflects a genuine representational limit.

A key structural finding is that successful progressive trajectories $\{e_1^*, e_2^*, \dots, e_{n^*}^*\}$ lie in a low-dimensional manifold in embedding space. PCA on these trajectories shows 80% of variance explained by 5 principal components—far fewer than the ambient embedding dimension of 4096—suggesting that token cramming exploits a structured low-dimensional code in the embedding space.

The Big Picture

Technical Formulation

Let the LLM f_θ be frozen with token vocabulary $\mathcal{V}$. Given prefix $x = (x_1, \dots, x_N) \in \mathcal{V}^N$, the cramming objective finds an embedding $e \in \mathbb{R}^d$:

$$e_n^* = \arg \min_{e \in \mathbb{R}^d} \mathcal{L}_{\text{recon}}(e, x_{1:n}), \quad \mathcal{L}_{\text{recon}} = \sum_{i=1}^n \ell_{\text{CE}}(f_\theta(e, x_{<i}), x_i)$$

subject to an optimization budget of B gradient steps. Progressive cramming solves this sequence for $n = 1, 2, \dots, N$, stopping at:

$$n^* = \max\{n : \mathcal{L}_{\text{recon}}(e_n^*, x_{1:n}) \leq \tau\}$$

The **compression horizon** is $\rho = n^*/N \in (0, 1]$: the fraction of the prefix representable within budget B .

Learning or Inference Procedure

After establishing compression horizons, the paper tests whether crammed embeddings e_n^* degrade task performance by prepending them to contexts where the full original prefix is also available:

1. Insert e^* as a soft prompt token before the original prefix.
2. Query the model on multiple-choice and open-ended generation tasks.
3. Compare accuracy to a baseline where no crammed embedding is inserted (original prefix only).

To locate the source of degradation, causal attention-knockout interventions systematically ablate attention edges between e^* and each layer’s query/key/value projections, measuring which ablation most recovers task performance.

What the Guarantee Says

There is no guarantee that a crammed embedding at the compression horizon n^* will not degrade task performance. The results show the opposite: even maximally compressed embeddings (at the representational limit) cause consistent accuracy drops. The horizon tells practitioners how much of a prefix the embedding *encodes*—but not how much it *transmits usefully to the model at inference time*.

Experimental Findings

Experiments use GPT-2-XL (1.5B), LLaMA-2-7B, and Mistral-7B as backbone models, with prefixes from C4 (general text), PubMed (medical), and HumanEval (code).

Task Type	No Crammed Embed.	+ Crammed Embed.
MMLU (4-choice)	63.4	56.7
HellaSwag (4-choice)	74.1	67.8
OpenGen (correctness)	51.3	18.2
Code completion	42.7	11.4

Compression horizons (fraction of 512-token prefixes representable): General text: 68%, Medical text: 55%, Code: 28%. Code is hardest to compress, consistent with the view that low-entropy, syntactically precise sequences exhaust embedding capacity fastest.

Ablations and Interpretation

Attention knockout: Ablating attention edges from e^* in layers 1–4 recovers 61% of the multiple-choice accuracy drop. Ablating in layers 12–16 recovers only 8%. Early layers are the primary site of destructive interaction.

Prefix vs. no prefix: Inserting e^* with the original prefix still degrades performance. Inserting e^* without the original prefix is catastrophic. This rules out the hypothesis that the embedding “blocks” the prefix from the model; rather, the embedding actively interferes with early attention distributions.

Budget sensitivity: Doubling B from 10^3 to 2×10^3 steps increases the compression horizon by only 3–6%, confirming that the horizon is not a function of insufficient optimization.

Dimension sensitivity: Increasing embedding dimension from $d = 4096$ to $d = 8192$ raises horizons by 11% on average, suggesting that embedding dimensionality is a partial bottleneck, though the gains plateau rapidly.

Reference

Dmitrii Tarasov, Timofei Lashukov, Elizaveta Goncharova, Andrey Kuznetsov. **Progressive Cramming: Reliable Token Compression and What It Reveals**. arXiv:2607.21231, July 23, 2026. <https://arxiv.org/abs/2607.21231>

Keywords: vision-language models • 3D spatial reasoning • implicit geometry • explicit geometry • ECCV 2026

3D-Aware VLMs with Implicit and Explicit Geometries

VLM-IE3D Equips Vision-Language Models with Two Complementary Geometric Representations Derived Purely from RGB Video, Setting New State-of-the-Art on Four 3D Spatial Benchmarks

By Wenhao Li, Xueying Jiang, Quanhao Qian, Deli Zhao, Ran Xu, Shijian Lu, Gongjie Zhang (Nanyang Technological University)

arXiv:2607.21595 • ECCV 2026 • July 23, 2026

Vision-language models have demonstrated impressive language and 2D visual understanding but struggle when tasks demand precise 3D spatial reasoning. A new ECCV 2026 paper from Nanyang Technological University introduces **VLM-IE3D**, which equips standard VLMs with two complementary 3D geometric representations—implicit and explicit—derived from RGB video alone, with no depth sensors or 3D supervision. The framework achieves state-of-the-art performance on 3D video detection, 3D visual grounding, 3D dense captioning, and spatial reasoning benchmarks.

AT A GLANCE

Problem: VLMs built on 2D visual pretraining lack the fine-grained 3D spatial understanding needed for grounding, detection, and spatial reasoning in 3D scenes.

Why it matters: 3D-capable VLMs are essential for robotics, AR/VR, and embodied AI, yet requiring depth sensors limits real-world deployment.

Method: Implicit Geometry Tokens (coarse global priors) and Explicit Geometry Tokens (fine-grained reconstructed structure) fused with 2D visual tokens via a 3D-aware adapter.

Result: State-of-the-art on ScanQA (+6.4 pp), ScanRefer (+7.7 pp), Scan2Cap (+8.5 CIDEr), and EmbodiedScan (+4.7 mAP).

Evidence: RGB-only at inference; ablations confirm both geometry types are necessary and complementary.

The Big Picture

Modern VLMs are trained on billions of image-text pairs, giving them powerful 2D visual understanding and broad world knowledge. Yet 3D spatial tasks—locating objects relative to each other in 3D space, predicting 3D bounding boxes, or answering questions about scene geometry—

remain a persistent weakness. These tasks require a model to reason about depth, surface orientation, and object extent that is only partially recoverable from 2D images.

Existing solutions typically assume access to depth sensors or LiDAR at inference time, which is unrealistic for camera-only deployments. VLM-IE3D bridges this gap: starting from ordinary RGB video, it derives two complementary geometric representations and injects them into the VLM as geometric token streams.

Why Existing Approaches Fall Short

Three prior strategies have been tried with limited success:

Monocular depth augmentation: Estimated depth maps are fed alongside RGB images. Depth estimation errors propagate to downstream tasks, and estimated depth lacks surface-normal precision required for grounding.

Implicit-only geometry: Some recent VLMs inject geometric priors learned from video pretraining via contrastive objectives. These provide coarse global layout but lack the local precision needed for fine-grained 3D grounding—a deficiency VLM-IE3D directly addresses with explicit geometry tokens.

Explicit-only geometry: Sparse point clouds from structure-from-motion (SfM) encode precise local structure but lack the holistic scene layout captured by implicit priors. Without the global context, reasoning about scene-level spatial relationships degrades.

VLM-IE3D combines both: implicit for global layout, explicit for local precision.

Core Mechanism

Implicit Geometry Tokens (IGTs) are produced by a lightweight video encoder that processes T frames jointly. The encoder attends to temporal motion parallax and relative depth ordering cues to produce a compact set of N_{IGT} global geometric tokens. These encode a coarse “3D cognitive map” of the scene: which regions are closer, rough surface orientations, and camera motion type.

Explicit Geometry Tokens (EGTs) are produced by running a differentiable SfM module on the input frames to recover a sparse point cloud $\mathcal{P} = \{(\mathbf{p}_i, \mathbf{n}_i)\}$. The point cloud is voxelized into a 3D feature grid and encoded by a sparse 3D convolutional encoder into N_{EGT} tokens. These encode precise object boundaries, surface normals, and depth discontinuities.

A **3D-aware adapter** fuses IGTs and EGTs with the VLM’s 2D visual tokens using cross-attention with geometry-conditioned masks that spatially align explicit geometric tokens with their 2D image counterparts.

Technical Formulation

Given video $V = \{f_1, \dots, f_T\}$, let $Z_{2D} \in \mathbb{R}^{N_{2D} \times d}$ be the 2D visual tokens from the VLM’s encoder. The IGT and EGT encoders produce:

$$Z_{\text{IGT}} = \phi_{\text{IG}}(V) \in \mathbb{R}^{N_{\text{IGT}} \times d}$$

$$Z_{\text{EGT}} = \phi_{\text{EG}}(\mathcal{P}(V)) \in \mathbb{R}^{N_{\text{EGT}} \times d}$$

The 3D-aware adapter computes fused tokens:

$$Z_{\text{fused}} = \text{Attn}(Q = Z_{2D}, K = V = [Z_{\text{IGT}}; Z_{\text{EGT}}; Z_{2D}]) + Z_{2D}$$

with a geometry-conditioned attention mask $\mathbf{M} \in \{0, -\infty\}^{N_{2D} \times (N_{\text{IGT}} + N_{\text{EGT}} + N_{2D})}$ where $M_{ij} = 0$ if EGT token j projects within spatial radius r of 2D token i , and $-\infty$ otherwise (no restriction on IGT tokens, which are global). The fused representation Z_{fused} concatenated with the text query tokens is passed to the frozen VLM backbone.

Learning or Inference Procedure

Training proceeds in two stages:

1. **Geometric pretraining:** ϕ_{IG} and ϕ_{EG} are pre-trained on 3D video datasets (ScanNet, ARKitScenes) with reconstruction objectives: IGTs supervised to predict surface orientation distributions, EGTs supervised via point cloud completion loss.
2. **End-to-end instruction tuning:** The full model (adapter + pretrained geometry encoders + frozen VLM) is fine-tuned on 3D instruction-following data with a standard language modeling loss.

At inference, only the RGB video is required. The SfM module runs at inference time using COLMAP, adding a modest overhead.

What the Guarantee Says

VLM-IE3D provides no formal guarantee on 3D accuracy, but the ablation results establish that removing either IGT or EGT degrades performance across all benchmarks—confirming that neither geometry type subsumes the other, and that the dual-geometry design is necessary rather than redundant.

Experimental Findings

Benchmark (metric)	Best Prior	VLM-IE3D
ScanQA (acc.)	46.3	52.7
ScanRefer (Acc@0.5)	41.2	48.9
Scan2Cap (CIDEr@0.5)	62.8	71.3
EmbodiedScan (mAP@0.25)	28.4	33.1

All results are with RGB-only input; prior state-of-the-art methods use ground-truth depth where noted.

Ablations and Interpretation

IGT only: Removes EGT. ScanRefer (grounding) degrades by 6.2 pp, confirming fine-grained local geometry is necessary for precise localization.

EGT only: Removes IGT. ScanQA (spatial reasoning) degrades by 4.1 pp, confirming coarse global layout is necessary for scene-level reasoning.

Naïve concatenation: Concatenates IGT and EGT without the geometry-conditioned mask. Average performance degrades by 3.8 pp, demonstrating that spatial alignment of explicit geometry tokens is critical.

SfM quality: Replacing COLMAP with a faster but noisier SfM variant reduces EmbodiedScan mAP by 2.3 pp, confirming a moderate sensitivity to point cloud quality.

Reference

Wenhao Li, Xueying Jiang, Quanhao Qian, Deli Zhao, Ran Xu, Shijian Lu, Gongjie Zhang. **3D-Aware VLMs with Implicit and Explicit Geometries**. arXiv:2607.21595, ECCV 2026. <https://arxiv.org/abs/2607.21595>

Keywords: large-scale training • mixture-of-experts • Ascend NPU • post-training • operations research AI

SLAI T-Rex: Full-Parameter Post-training of the DeepSeek-V4 Family on Ascend SuperPOD

Huawei Cloud Achieves a $2.93\times$ MFU Improvement Over the GPU Baseline for Trillion-Parameter MoE Post-Training on Ascend NPUs, Demonstrating Hardware-Agnostic Frontier LLM Training

By SLAI Team (Huawei Cloud AI Lab)

arXiv:2607.20145 • July 2026

Training frontier large language models has been synonymous with NVIDIA GPU clusters. A new technical report from Huawei Cloud challenges this assumption: SLAI T-Rex demonstrates that full-parameter post-training of DeepSeek-V4—a 671-billion-parameter Mixture-of-Experts model—can be performed efficiently on Ascend NPU SuperPOD clusters via a hierarchical optimization framework spanning model parallelism, communication scheduling, and low-level kernel engineering. The system achieves 34.22% Model FLOPs Utilization (MFU)—a $2.93\times$ improvement over the open-source GPU baseline—and produces an operations-research specialist model that outperforms GPT-5.4-Mini.

AT A GLANCE

Problem: Full-parameter post-training of trillion-parameter MoE models on non-GPU hardware suffers from memory pressure, communication overhead, and kernel inefficiency.

Why it matters: Hardware diversity in large-scale LLM training reduces dependency on a single chip vendor and expands the ecosystem of frontier model development.

Method: Expert Shard Parallelism, communication-aware scheduling, and Ascend-specific CANN kernel optimizations, combined into a three-level hierarchical framework.

Result: 34.22% MFU ($2.93\times$ over baseline); OR-specialist model achieves 71.81% zero-shot Pass@1, +3.98 pp over GPT-5.4-Mini.

Evidence: Validated on DeepSeek-V4-Flash (671B active params) across a full CPT+SFT post-training workflow.

The Big Picture

DeepSeek-V4 is a Mixture-of-Experts model with 671 billion total parameters (37 billion active per token), trained with 4096 experts grouped into routed and shared expert layers. Post-training this model from scratch on Ascend hardware requires solving problems at three intersecting layers: how to distribute the model’s enormous weight across memory-constrained NPU nodes, how to orchestrate the expert-parallel all-to-all communications that differ fundamentally from NVLink-based topologies, and how to write efficient NPU kernels for the specific operations (gated attention, top- k routing, grouped GEMM) that account for the bulk of compute time.

Why Existing Approaches Fall Short

The open-source DeepSeek-V4 training recipe was designed for NVIDIA GPU clusters with NVLink interconnects. Porting it to Ascend SuperPOD directly yields only $\sim 11.7\%$ MFU because:

- Ascend’s ScaleUp network has different bandwidth/latency characteristics than NVLink, making the GPU pipeline schedule suboptimal.
- Expert parallelism requires all-to-all communication patterns that expose the Ascend cluster’s inter-pod bandwidth as a bottleneck.
- Standard CUDA-derived kernels are mapped inefficiently to Ascend’s cube tile architecture, leaving the NPU’s SIMD units underutilized.

Core Mechanism

SLAI T-Rex addresses these challenges through a **three-level hierarchical optimization framework**:

Level 1 — Model Parallelism: Expert Shard Parallelism (ESP) shards each expert’s weight matrix across multiple NPU pods, reducing per-device peak memory. A modified pipeline schedule overlaps expert computation with inter-pod communication by reordering micro-batches to ensure that communication and compute streams are never both idle.

Level 2 — Communication Orchestration: A communication-aware scheduler predicts expert load imbalance from recent routing statistics, adjusts micro-batch boundaries to reduce hot-expert serialization, and overlaps all-to-all token dispatches with local attention computations via asynchronous execution streams.

Level 3 — Kernel Engineering: Custom CANN kernels implement fused gating (softmax + top- k selection), Flash Attention adapted to Ascend’s HBM tile layout, and gradient all-reduce with dynamic bucketing tuned to Ascend’s memory bandwidth profile.

Technical Formulation

For an MoE layer with E experts and top- k routing, the routing gate selects:

$$g(x) = \text{TopK}(\text{softmax}(W_g x), k)$$

and the layer output is:

$$\text{MoE}(x) = \sum_{e \in \text{TopK}(x)} g_e(x) \cdot \text{FFN}_e(x)$$

Expert parallel training communicates $O(B \cdot k \cdot d)$ token embeddings per step (where B is batch size and d is hidden dimension) across all expert pods. The SLAI framework fuses the dispatch (input routing) and combine (output aggregation) all-to-all operations and prefetches expert weights from HBM, reducing effective communication latency by 38% compared to the baseline.

MFU is defined as the ratio of observed compute to peak theoretical compute:

$$\text{MFU} = \frac{\text{Observed TFLOPs/sec}}{\text{Peak Theoretical TFLOPs/sec (Ascend cluster)}}$$

The SLAI T-Rex system achieves $\text{MFU} = 34.22\%$ on DeepSeek-V4-Flash post-training, compared to 11.68% for the open-source GPU recipe adapted naively to Ascend.

Learning or Inference Procedure

The OR-specialist model development follows a two-stage workflow:

1. **Continued Pretraining (CPT):** DeepSeek-V4-Flash is continued-pretrained on a curated corpus of operations research literature, textbooks, LP/MILP problem libraries, and synthetic problem-solution pairs.
2. **Supervised Fine-tuning (SFT):** 10,000 high-quality SFT samples spanning four task categories (LP, MILP, scheduling, routing) and three problem representations (mathematical, natural language, structured JSON) are used to align the model’s output format and reasoning style.

The final model is evaluated zero-shot at Pass@1 without any inference-time search.

What the Guarantee Says

The paper does not provide theoretical guarantees on MFU or task accuracy. The MFU improvement is empirically validated on a full-scale DeepSeek-V4-Flash post-training run, and the $2.93\times$ figure represents end-to-end wall-clock throughput improvement including all communication and memory overhead.

Experimental Findings

Model	MFU	OR Pass@1
GPU baseline (open-source)	11.68%	—
SLAI T-Rex	34.22%	—
DeepSeek-V4-Flash (base)	—	60.54%
GPT-5.4-Mini	—	67.83%
SLAI T-Rex OR-specialist	—	71.81%

The $2.93\times$ MFU improvement translates directly to proportionally reduced wall-clock training time and therefore training cost on the same hardware budget. The OR-specialist achieves highest zero-shot accuracy among evaluated models on a held-out OR benchmark spanning diverse problem types and sizes.

Ablations and Interpretation

ESP alone vs. full framework: Expert Shard Parallelism alone raises MFU from 11.68% to 19.4%; adding the communication scheduler raises it to 27.1%; adding Ascend-specific kernels reaches 34.22%.

CPT vs. SFT-only: The OR-specialist without CPT (SFT-only on the base model) achieves 66.3% Pass@1, 5.5 pp below the full CPT+SFT model, confirming that domain vocabulary and mathematical notation pretraining is material.

Data representation: Models trained on all three problem representations outperform single-format training by 2.1–3.8 pp, suggesting that diverse input format exposure improves generalization.

Reference

SLAI Team. **SLAI T-Rex: Full-Parameter Post-training of the DeepSeek-V4 Family on Ascend SuperPOD**. arXiv:2607.20145, July 2026. <https://arxiv.org/abs/2607.20145>

Keywords: video generation • diffusion models • multi-agent simulation • world models • autonomous driving

Streaming Multi-Agent Autoregressive Diffusion Model with World State Registers

A Shared World State Register Enables Arbitrarily Long, Spatially Consistent Multi-Perspective Video Generation Without Quadratic Context Growth, Advancing Autonomous Driving Simulation

By Sicheng Mo, Yuheng Li, Ziyang Leng, Krishna Kumar Singh, Bolei Zhou (University of California, Los Angeles)

arXiv:2607.21594 • July 23, 2026

Autonomous driving simulators, multi-agent games, and embodied AI research all require generating photorealistic video from multiple agent perspectives that remain mutually consistent over long time horizons. Current video diffusion models generate single-agent views or suffer from consistency drift when extended across multiple agents and timesteps. A new paper from UCLA introduces a **Streaming Multi-Agent Autoregressive Diffusion (SMAD)** framework with **World State Registers (WSR)**—a fixed-size shared latent mechanism that aggregates and broadcasts global world state to all agents at each step, reducing Fréchet Video Distance by 29% and improving cross-agent consistency by 34% over strong baselines.

AT A GLANCE

Problem: Multi-agent video generation with independent per-agent diffusion models produces mutually contradictory world states that diverge over long rollouts.

Why it matters: Photorealistic, coherent multi-agent simulation is a critical infrastructure component for autonomous driving and embodied AI training at scale.

Method: Streaming autoregressive diffusion per agent, conditioned on a shared World State Register updated each step via a write-then-broadcast mechanism.

Result: FVD of 294 vs. 412 (independent baseline) and 0.82 vs. 0.61 cross-agent consistency score.

Evidence: Evaluated on a 6-camera surround autonomous driving benchmark across 100-step rollouts; ablations confirm WSR update frequency and register size are critical parameters.

The Big Picture

Training autonomous vehicles, robots, and embodied agents at scale requires generating realistic sensor data—particularly video—from multiple co-located agents simultaneously. The gold standard is a world model that produces photorealistic, physically plausible multi-perspective video for any number of agents over arbitrarily long time horizons, with all agent views remaining mutually consistent.

Standard video diffusion models address single-agent scenarios. Naïve extension to multi-agent settings either (a) generates each agent’s view independently—producing contradictory depictions of shared world elements—or (b) concatenates all agents’ contexts as conditioning information, incurring memory costs that scale as $O(A \cdot T \cdot d)$ (agents $\times$ timesteps $\times$ dimension), making long rollouts intractable.

Why Existing Approaches Fall Short

Independent per-agent diffusion: Without cross-agent coordination, two agents viewing the same intersection from different angles may generate inconsistent traffic light states, object positions, or pedestrian motions. These inconsistencies compound over time.

Full history concatenation: Conditioning each agent’s diffusion model on all other agents’ full history provides consistency at the cost of quadratic memory growth. For 6 cameras $\times$ 100 timesteps, this is computationally prohibitive.

Fixed-frame world model: Existing multi-camera driving world models generate fixed-length video clips rather than streaming generation, making them unsuitable for online simulation.

SMAD addresses all three limitations through the WSR mechanism.

Core Mechanism

The **World State Register** is a compact set of M latent vectors $\mathbf{s} \in \mathbb{R}^{M \times d}$ that encodes the current global state of the simulated world. At each generation step t :

1. Each of the A agents generates its next frame conditioned on its local sliding window and the current register $\mathbf{s}_t$.
2. The register is **updated** via cross-attention over all agents’ newly generated frame encodings.
3. The updated register $\mathbf{s}_{t+1}$ is broadcast to all agents for step $t + 1$.

Because the register has fixed size M (independent of A and T), per-agent memory cost is constant regardless of simulation length or the number of agents. The register serves as a sufficient statistic for cross-agent coordination:

each agent needs only the register, not other agents’ raw frames.

Technical Formulation

The joint generative distribution factorizes as:

$$p(x_{1:T}^{(1:A)}) = \prod_{t=1}^T p(\mathbf{s}_t | x_{<t}^{(1:A)}, \mathbf{s}_{t-1}) \prod_{a=1}^A p_\theta(x_t^{(a)} | x_{t-W:t-1}^{(a)}, \mathbf{s}_t)$$

where W is the local history window size. Each per-agent model p_θ is a latent diffusion model with the register injected as cross-attention conditioning:

$$x_t^{(a)} \sim p_\theta(\cdot | x_{t-W:t-1}^{(a)}, \mathbf{s}_t, c^{(a)})$$

where $c^{(a)}$ encodes agent-specific conditioning (camera pose, trajectory, goal). The register update is:

$$\mathbf{s}_{t+1} = \text{Attn}(Q = \mathbf{s}_t, K = V = [\mathbf{s}_t; \phi_{\text{enc}}(x_t^{(1)}); \dots; \phi_{\text{enc}}(x_t^{(A)})])$$

A register compression loss ensures the register faithfully captures world state:

$$\mathcal{L}_{\text{reg}} = \mathbb{E} \left[\left\| \mathbf{s}_t - \phi_{\text{enc}}(x_t^{(1:A)}) \right\|^2 \right]$$

added to the diffusion denoising objective during training.

Learning or Inference Procedure

Training uses paired multi-agent driving video from the nuScenes-derived MultiAgent benchmark (6 surround cameras, synchronized at 12 fps). The per-agent diffusion model and register update module are trained jointly end-to-end. The register is initialized to a learned null vector at the start of each sequence and is updated autoregressively without backpropagating through the full rollout history (stopped gradient on $\mathbf{s}_{t-1}$ during training).

At inference, generation proceeds streaming: no future frames are required, and the register can be updated in real time as frames are generated, enabling online simulation.

What the Guarantee Says

The factorization structure guarantees that the register is the *only* information channel between agents at each step. This means that any information not captured in the M -vector register (e.g., fine-grained local details beyond what the update attention can aggregate) cannot be communicated across agents. Register size M thus controls the fidelity–efficiency tradeoff.

Experimental Findings

Method	FVD ↓	Consistency ↑
Independent per-agent	412	0.61
Shared history concat.	387	0.68
SMAD (WSR)	294	0.82

Long-horizon stability (normalized score at 100 steps) is 0.74 for SMAD vs. 0.43 for independent baselines, demonstrating that the WSR effectively prevents temporal drift over long rollouts.

Ablations and Interpretation

WSR update frequency: Updating the register every step vs. every 4 steps degrades FVD from 294 to 361, confirming that frequent synchronization is necessary for maintaining consistency.

Register size M : FVD plateaus at $M = 64$ vectors. Below $M = 32$, cross-agent consistency collapses; above $M = 128$, there is no measurable benefit.

Local window W : Replacing the local $W = 8$ frame window with global history improves FVD marginally (288 vs. 294) but increases per-step memory by $12\times$, confirming the local window is an efficient approximation.

Register initialization: Random initialization produces $\Delta\text{FVD} = +14$ vs. the learned null vector, confirming that a well-initialized register provides important context for the first few generation steps.

Reference

Sicheng Mo, Yuheng Li, Ziyang Leng, Krishna Kumar Singh, Bolei Zhou. **Streaming Multi-Agent Autoregressive Diffusion Model with World State Registers**. arXiv:2607.21594, July 23, 2026. <https://arxiv.org/abs/2607.21594>